

PROJECT WRITE-UP

MITRE ATT&CK Chain Simulator

Simulating Adversary Attack Chains, Mapping Techniques to MITRE ATT&CK, and Evaluating Defensive Detection Controls

Project Detail	Description
Project Category	Cybersecurity / Threat Detection & Adversary Simulation
Environment	Single-page browser application, local browser or GitHub Pages deployment
Primary Technologies	HTML, CSS, JavaScript, DOM manipulation, browser-based random detection model
Core Functions	Scenario selection, security-control toggles, stage-by-stage simulation, sensor logging, tactic results
Outcome	A self-contained defensive simulation tool that demonstrates ATT&CK-oriented attack-chain modeling and control coverage analysis

Project source - github.com/calebja/MITRE_ATT-CK_Simulator

Table of Contents

1. Executive Summary
2. Problem Statement / Objectives
3. Application Architecture & Environment
 - 3.1 Application Structure
 - 3.2 Threat Modeling Structure
4. Tools & Technologies
5. Methodology
 - 5.1 Defining the Problem and Scope
 - 5.2 Modeling the Attack Scenarios
 - 5.3 Modeling Defensive Controls
 - 5.4 Building the User Interface
 - 5.5 Implementing the Simulation Engine
 - 5.6 Aggregating and Reporting Results
 - 5.7 Safety, Validation & Deployment
6. Results
7. Detection Coverage & Risk Interpretation
8. Lessons Learned
9. Deliverables & Impact
10. Conclusion
- Appendix A - Implementation Reference
- Appendix B - ATT&CK Scenario Mapping

1. Executive Summary

This project documents the design and construction of a browser-based MITRE ATT&CK Chain Simulator. The goal was to turn the abstract idea of an adversary attack chain into an interactive, repeatable learning exercise in which a user can select a scenario, choose which defensive controls are available, run a simulated chain, and immediately see which stages would have generated an alert.

The application deliberately avoids real attack execution, and there is no real network, host, or account. Stage outcomes are calculated in the browser from a simple detection-probability model, and the project contains no exploit code or external calls. The implementation is a single static HTML file with embedded CSS and JavaScript, which keeps the tool easy to inspect, run locally, and publish through GitHub Pages. [1]

The finished simulator models three example scenarios (phishing-led compromise, ransomware staging, and insider misuse) and six security controls. These include an email security gateway, EDR/endpoint telemetry, MFA on privileged accounts, SIEM log correlation, network firewall/IDS, and DLP/egress monitoring. Each scenario follows a six-stage chain from Phishing through Data Collection, with a technique identifier and ATT&CK tactic associated with each stage. [1] [2] [3]

The primary outcome was not a claim that the application predicts real-world detection performance. Instead, the project produced a practical teaching and portfolio artifact showing how threat modeling, defensive control coverage, JavaScript application logic, user-interface design, and ATT&CK-oriented reporting can be connected in one small, self-contained security project.

2. Problem Statement / Objectives

Security teams frequently describe attacks as sequences of tactics and techniques, while defensive teams think in terms of controls, telemetry, and alerts. I wanted a project that connected those two viewpoints in a form that was easier to experiment with than a static ATT&CK matrix or a written incident scenario. The problem was therefore to create a safe simulation that makes the relationship between an attack stage and a defensive control visible without executing real attacks.

- Create a self-contained web application that can run without a build system, package manager, backend, or external service.
- Represent multiple attack-chain scenarios using MITRE ATT&CK-aligned tactic and technique identifiers.
- Allow the user to toggle defensive controls on and off before running a scenario.
- Simulate stage-level detection so that the same scenario can produce different outcomes based on the selected control stack.
- Show the attack chain progressing visually and record which control, if any, generated an alert at each stage.
- Aggregate individual stage outcomes into tactic-level results that can be read quickly as a coverage-gap summary.
- Keep the project explicitly non-operational and safe for a portfolio, classroom, or home-lab learning environment.

3. Application Architecture & Environment

The simulator is intentionally small. The application consists of an index.html file that holds the markup, styles, and simulation logic, with no build step or runtime dependencies. It can be opened directly in a browser or deployed to GitHub Pages. [1]

3.1 Application Structure

Component	Implementation	Purpose
Scenario data	JavaScript SCENARIOS array	Defines the three example scenarios, stage labels, ATT&CK technique IDs, tactics, and control probabilities.
Control data	JavaScript CONTROLS array	Defines the six simulated security controls displayed to the user.
Scenario selector	HTML + DOM rendering	Lets the user switch between the phishing-led, ransomware, and insider scenarios.
Control toggles	HTML + JavaScript state	Enables or disables simulated controls before a run.
Attack chain view	HTML + CSS + DOM state classes	Displays each stage sequentially and marks it as detected or undetected.
Sensor log	JavaScript DOM updates	Records the active stage and the control that raised an alert, or records that no enabled control detected it.
Results table	JavaScript aggregation	Groups outcomes by tactic and displays a final detected/undetected result.

Table 1. Major components of the single-page simulator.

3.2 Threat Modeling Structure

Each scenario is represented as six ordered stages. The current project uses the following conceptual flow.

```
Phishing -> Credential Theft -> Initial Access -> Privilege Escalation -> Lateral Movement -> Data Collection
```

The project intentionally describes its ATT&CK mappings as "loosely" mapped to tactics and technique IDs. The project is demonstrating the structure of threat-informed detection modeling, not reproducing a particular threat actor operation or claiming procedure-level fidelity to ATT&CK case data. [1] MITRE ATT&CK defines tactics as the adversary goal behind a technique and techniques as the actions used to achieve that goal. [2] [3] The simulator uses that tactic-oriented framing for its final reporting.

4. Tools & Technologies

Technology / Tool	Role in Project
HTML5	Defines the application structure, scenario/control panels, attack-chain display, console, and results table.
CSS3	Provides the responsive dark cyber-range presentation, card layout, state styling, and visual indicators for hits and misses.
JavaScript	Implements state management, scenario/control rendering, asynchronous stage progression, random detection checks, logging, and results aggregation.
DOM API	Creates and updates scenario buttons, control toggles, chain stages, sensor-log entries, and results rows dynamically.

Technology / Tool	Role in Project
GitHub / GitHub Pages	Provides source control and a simple static deployment target for the completed application.
MITRE ATT&CK	Provides the threat-informed vocabulary used for tactics and technique identifiers associated with scenario stages.
Browser Math.random()	Provides the stochastic roll used by the demonstration detection model for each enabled control at each stage.

5. Methodology

5.1 Defining the Problem and Scope

The first design decision was to keep the project educational and defensive rather than operational. The scope was limited to representing a chain and estimating whether a simulated control would detect each stage. No packet generation, credential access, exploitation, command execution on remote systems, or external API integration was included. This reduced the project to a controllable modeling problem consisting of an attack stage, enabled controls, detection probabilities, and a simulated alert outcome.

I also chose a static architecture so that the entire application could be understood by reading one file. This reduced setup friction and made it possible to host the finished project as a simple GitHub Pages site. The application documentation confirms that it can be opened locally or served directly as a static page. [1]

5.2 Modeling the Attack Scenarios

The scenario model was built around three different storylines so that the same defensive control stack could be tested against different forms of attacker behavior.

Scenario	Scenario Intent	Six-Stage Flow
Phishing-led compromise	A lure email harvests credentials and the attacker pivots into the environment.	Phishing -> Credential Theft -> Initial Access -> Privilege Escalation -> Lateral Movement -> Data Collection
Ransomware staging	A dropper enters through a service, progresses through the environment, and reaches data impact/collection stages.	Phishing -> Credential Theft -> Initial Access -> Privilege Escalation -> Lateral Movement -> Data Collection
Insider misuse	A valid user account is abused for slower, quieter data staging.	Phishing -> Credential Theft -> Initial Access -> Privilege Escalation -> Lateral Movement -> Data Collection

Table 2. Scenario model defined in the project source.

The important implementation choice was to store each scenario as data rather than hard-code a separate execution function for every storyline. Each stage records its label, technique identifier, tactic, and a set of control-specific detection probabilities. That design makes the model easy to extend. Additional scenarios can be added by inserting another data object rather than rewriting the simulation engine. [1]

5.3 Modeling Defensive Controls

Control	What the simulator represents
Email security gateway	A mail-layer detection/prevention point, primarily relevant to phishing activity.
EDR / endpoint telemetry	Endpoint visibility that can detect credential theft, privilege escalation, and related host activity.
MFA on privileged accounts	An identity control that can reduce the chance of successful use of stolen credentials.
SIEM log correlation	Cross-source logging/correlation that can identify suspicious behavior across stages.
Network firewall / IDS	Network monitoring intended to detect certain ingress and lateral-movement behaviors.
DLP / egress monitoring	Data-movement monitoring intended to detect collection or attempted transfer of sensitive data.

The default state in the source enables email security, EDR, MFA, SIEM, and firewall/IDS while leaving DLP disabled. The user can change that state before a simulation run. [1] CISA recommends requiring multifactor authentication where possible and starting with privileged or administrative accounts, which provides context for the simulator's MFA control. [7]

5.4 Building the User Interface

The interface was organized into two functional columns. The left side contains scenario selection and control toggles. The right side contains the attack chain, sensor log, and final results. This mirrors the workflow a user follows. Configure the exercise first, then observe the simulated execution and review the defensive outcome.

Visual state is communicated through CSS classes. A stage becomes active while it is being processed, then receives a separate hit or miss state after the detection check. The results table uses a compact yes/no-style status so the user can scan the final tactic-level coverage quickly. [1]

5.5 Implementing the Simulation Engine

The simulation engine iterates through the selected scenario one stage at a time. For each stage, the application writes the technique identifier to the sensor log, pauses briefly to make the progression visible, checks each configured control that is currently enabled, and uses `Math.random()` against the control probability to determine whether an alert fires. The first successful detection ends the control-check loop for that stage.

```
for each stage in selected scenario:
    mark stage as active
    write stage + technique ID to sensor log
    for each configured control:
        skip control when disabled
        generate random roll
        if roll < configured probability:
            mark stage DETECTED and log the control
            stop checking controls for this stage
    otherwise mark stage UNDETECTED
```

This implementation keeps the logic deterministic in structure but stochastic in outcome. The sequence of stages is fixed, while the detection result can change between runs because the browser generates a new random value for each enabled-control check.

5.6 Aggregating and Reporting Results

After all six stages finish, the simulator groups the individual outcomes by tactic. The source also supports a fold property so that a sub-stage can be merged into the tactic row it names. In the current scenarios, the Phishing stage is folded into the Initial Access result so that the final table reports a concise tactic-level view rather than treating every stage as an independent tactic row. [1] NIST developed SP 800-92 to help organizations develop, implement, and maintain effective log management practices, which provides context for the simulator's sensor log and SIEM control. [6]

The final rate is calculated from the number of tactic rows detected divided by the number of tactic rows in the final report. The interface explicitly frames an undetected result as a control-coverage gap for the current scenario, which makes the output useful for discussing defensive blind spots without implying that a single simulation is a real SOC performance metric. [1]

5.7 Safety, Validation & Deployment

Safety was treated as a product requirement rather than an afterthought. The project includes a visible notice stating that the environment is simulated, that nothing touches a real network, host, or account, and that outcomes are calculated by a detection-probability model rather than executed attack code. The implementation contains no attack or exploit code and makes no external calls. [1] NIST SP 800-61 Rev. 3 places incident response within broader cybersecurity risk management and addresses improvements to detection, response, and recovery activities. [5]

Validation focused on the user-visible lifecycle. Scenario selection should update the chain, control toggles should change state, the Run button should step through all stages, the sensor log should reflect detections and misses, and the results table should appear only after the run completes. Because the project is static, deployment validation also means confirming that the same index.html works when opened directly and when served from GitHub Pages. [1]

6. Results

The completed project meets the core functional goal. A user can choose among three scenarios, configure six simulated security controls, execute a six-stage attack chain, observe stage-by-stage logging, and receive a tactic-level detection summary. The implemented example results show Initial Access, Credential Access, and Lateral Movement detected while Privilege Escalation and Data Collection are not. This demonstrates the intended output format as a compact representation of where the configured controls did and did not provide coverage. [1]

6.1 Implemented Feature Results

Feature	Result	Evidence in Implementation
Scenario selection	Implemented	Three scenario objects rendered into selectable buttons.
Control toggles	Implemented	Six controls rendered with on/off state and disabled during a run.
Six-stage attack chain	Implemented	Stages rendered dynamically with technique IDs and sequential state changes.
Sensor log	Implemented	Each stage writes either the detecting control or an undetected message.
Tactic-level results	Implemented	Stage outcomes are grouped and displayed in a final results table.
Detection coverage rate	Implemented	Final report calculates detected tactic rows / total tactic rows.
Static deployment	Implemented	Single index.html with no build step or dependencies, compatible with GitHub Pages.
Safe simulation boundary	Implemented	Explicit notice, no real systems, attack code, or external calls.

Table 3. Functional results from review of the project implementation.

6.2 Default Detection Model - Implementation-Derived Probabilities

The following values are not measured SOC detection rates. They are the single-stage probabilities implied by the current code when the application loads with its default control state. Email, EDR, MFA, SIEM, and firewall/IDS are enabled, while DLP is disabled. For stages with multiple enabled controls, the probability of at least one detection is computed from the sequential checks in the source. This provides a useful way to understand how the configuration affects the simulation. [1]

Scenario	Initial Access*	Credential Access	Privilege Escalation	Lateral Movement	Data Collection
Phishing-led compromise	93.8%	58.8%	61.0%	67.0%	20.0%
Ransomware staging	86.0%	68.0%	47.5%	54.5%	30.0%
Insider misuse	52.8%	20.0%	44.0%	51.3%	25.0%

* Initial Access combines the Phishing and Initial Access stages using the simulator's fold/aggregation behavior.

These values show why the same control stack can behave differently across scenarios. The set of relevant controls and probabilities changes with the modeled behavior. They also explain why repeating the simulation can produce different visible outcomes even when the configuration does not change.

6.3 Representative Results Flow

Scenario selected: Phishing-led compromise

```
Controls online: Email, EDR, MFA, SIEM, Firewall/IDS
DLP: OFF
Phishing          -> alert / no alert
Credential Theft  -> alert / no alert
Initial Access    -> alert / no alert
Privilege Escalation -> alert / no alert
Lateral Movement -> alert / no alert
Data Collection   -> alert / no alert
Final report      -> tactic-level detection summary
```

7. Detection Coverage & Risk Interpretation

The most important security lesson from the project is that detection coverage is scenario-dependent. Enabling more controls generally creates more opportunities for a stage to be detected, but the simulator intentionally does not model a universal "good" or "bad" control stack. Instead, it demonstrates that a control only helps when the modeled stage has a relationship to that control, and that a gap at one stage can remain even when several other stages are visible.

Observed Design Pattern	Interpretation for a Security Team
Initial Access benefits from identity + email controls in the phishing scenario.	Layered controls can provide multiple points at which an initial compromise may become visible or be interrupted.
Credential Theft relies heavily on EDR and SIEM in the modeled scenarios.	Endpoint telemetry and centralized correlation are complementary rather than interchangeable in the simulation.
Lateral Movement uses firewall/IDS and SIEM probabilities.	Network visibility is represented as one part of a broader detection strategy, not as the sole control.
Data Collection remains comparatively exposed when DLP is disabled.	The model makes the effect of removing an egress/data-control layer easy to visualize.
Each run can change because detection is random.	A single run should be treated as a demonstration, not as a statistical estimate of real control efficacy.

The broader value of this exercise is therefore analytical rather than operational. It provides a simple interface for asking, "Where would this behavior become visible with this control stack?" That question is directly aligned with threat-informed defensive thinking and ATT&CK-oriented detection engineering. [2] NIST CSF 2.0 provides a common structure for managing and communicating cybersecurity risk, and this detection-focused exercise fits within that broader risk management context. [4] CISA also recommends enabling logs on endpoint devices and firewalls and centralizing logs to support detection of unusual activity, which is reflected conceptually in the simulator's SIEM and network monitoring controls. [8]

8. Lessons Learned

- Threat modeling becomes easier to communicate when the attack sequence is represented as data. The SCENARIOS structure separates scenario content from the engine that executes it.
- A defensive control should be tied to a specific behavior rather than treated as a generic detection layer. The per-stage probability objects make those relationships explicit.
- Simple browser technologies are sufficient for a useful cybersecurity training artifact. HTML, CSS, and JavaScript can represent stateful simulations without a backend.

- Randomized outcomes are useful for demonstrating uncertainty, but they reduce repeatability. A future test mode should provide deterministic or seeded runs for validation and demonstrations.
- Good security tooling needs to communicate its own limits. The simulation notice is important because the application visualizes attack concepts without performing real attacks.
- The final result is more useful when raw stage outcomes are aggregated into a decision-oriented report. The tactic folding behavior turns six stage checks into a compact coverage summary.

9. Deliverables & Impact

The project produced a reusable, portfolio-ready web application rather than only a code exercise. The result is a single static file that can be inspected, run locally, or deployed as a static web page, which makes it easy for another student, instructor, or interviewer to access the demonstration without installing a Python environment, database, or security framework. [1]

- Interactive MITRE ATT&CK-oriented attack-chain simulator with three scenarios.
- Six simulated defensive controls with configurable on/off state.
- Stage-by-stage visual progression and sensor log output.
- Tactic-level detection summary and coverage-gap reporting.
- Explicit safe-use boundary showing that no real network or host is touched.
- Hosted source code with a minimal setup path suitable for static deployment.
- A project artifact that demonstrates cybersecurity concepts alongside front-end development and JavaScript application design.

For a cybersecurity portfolio, the project demonstrates several distinct skills at once. These include translating a security concept into a software model, mapping behaviors to ATT&CK language, reasoning about defensive controls, implementing interactive state, and documenting security limitations clearly. The underlying implementation also exposes the logic directly, making the work reviewable rather than presenting only a finished interface. [1]

10. Conclusion

This project achieved its core objective of turning a threat-informed detection concept into a concrete, interactive application. The completed MITRE ATT&CK Chain Simulator combines three example adversary storylines, six defensive controls, a six-stage attack-chain view, sensor-style logging, and tactic-level results in a single browser page. The code keeps the environment deliberately safe by modeling detection outcomes instead of executing attacks. [1]

The most useful outcome was the bridge between offensive behavior and defensive visibility. A user can change the control stack, run the same scenario again, and observe how coverage changes. That makes the project suitable as a learning aid, portfolio artifact, and foundation for future work such as deterministic test runs, multi-run analytics, ATT&CK technique detail panels, exportable results, or additional scenarios.

Appendix A - Implementation Reference

A.1 Control Definitions

```
email -> Email security gateway
edr   -> EDR / endpoint telemetry
mfa   -> MFA on privileged accounts
siem  -> SIEM log correlation
fw    -> Network firewall / IDS
dlp   -> DLP / egress monitoring
```

A.2 Example Scenario Object

```
{
  id: 'phish',
  name: 'Phishing-led compromise',
  stages: [
    { label: 'Phishing', tech: 'T1566', tactic: 'Initial Access', ... },
    { label: 'Credential Theft', tech: 'T1056', tactic: 'Credential Access', ... },
    { label: 'Initial Access', tech: 'T1078', tactic: 'Initial Access', ... },
    ...
  ]
}
```

A.3 Core Detection Logic

```
for (const [controlId, probability] of Object.entries(stage.use)) {
  if (!on[controlId]) continue;
  const roll = Math.random();
  if (roll < probability) {
    detected = true;
    hitControl = control.name;
    break;
  }
}
```

A.4 Results Aggregation

```
const key = outcome.tactic;
if (!(key in byTactic)) byTactic[key] = false;
byTactic[key] = byTactic[key] || outcome.detected;
```

In the current implementation, an outcome remains true for a tactic once any contributing stage has been detected. The fold property allows the scenario author to decide which stages should roll up into the same reporting row. [1]

Appendix B - ATT&CK Scenario Mapping

The table below reproduces the technique identifiers and tactic labels defined in the current project source. They should be read as the simulator author's training-oriented mappings, not as a claim that every stage is a complete ATT&CK procedure implementation. [1]

Scenario	Stage	Technique ID	Tactic
----------	-------	--------------	--------

Scenario	Stage	Technique ID	Tactic
Phishing-led compromise	Phishing	T1566	Initial Access
Phishing-led compromise	Credential Theft	T1056	Credential Access
Phishing-led compromise	Initial Access	T1078	Initial Access
Phishing-led compromise	Privilege Escalation	T1068	Privilege Escalation
Phishing-led compromise	Lateral Movement	T1021	Lateral Movement
Phishing-led compromise	Data Collection	T1005	Data Collection
Ransomware staging	Phishing	T1190	Initial Access
Ransomware staging	Credential Theft	T1003	Credential Access
Ransomware staging	Initial Access	T1133	Initial Access
Ransomware staging	Privilege Escalation	T1055	Privilege Escalation
Ransomware staging	Lateral Movement	T1570	Lateral Movement
Ransomware staging	Data Collection	T1486	Data Collection
Insider misuse	Phishing	-	Initial Access
Insider misuse	Credential Theft	T1552	Credential Access
Insider misuse	Initial Access	T1078.004	Initial Access
Insider misuse	Privilege Escalation	T1548	Privilege Escalation
Insider misuse	Lateral Movement	T1021.002	Lateral Movement
Insider misuse	Data Collection	T1213	Data Collection

Table 4. Technique and tactic mappings as implemented in index.html.

Source Notes

-
- [1] Project source files and documentation - github.com/calebja/MITRE_ATT-CK_Simulator
 - [2] MITRE ATT&CK Enterprise Tactics - attack.mitre.org/tactics
 - [3] MITRE ATT&CK Enterprise Techniques - attack.mitre.org/techniques
 - [4] NIST Cybersecurity Framework 2.0 - nist.gov/cyberframework
 - [5] NIST SP 800-61 Rev. 3 Incident Response - csrc.nist.gov/pubs/sp/800/61/r3/final
 - [6] NIST SP 800-92 Guide to Computer Security Log Management - csrc.nist.gov/pubs/sp/800/92/final
 - [7] CISA Require Multifactor Authentication - cisa.gov/require-multifactor-authentication
 - [8] CISA Use Logging on Business Systems - cisa.gov/use-logging-on-business-systems